

Patient Reallocation and Waitlist Reduction in the Norwegian GP System

Ikke hvor mange, men hvilke?

26.06.2026

Lars Møen Haukland

Contents



1	Intro	4
1.a	Motivasjon	5
1.b	Løsningen?	7
1.c	Bakgrunn	9
1.d	Top Trading Cycles (TTC)	10
1.e	Huitfeldt et al. TTC	12
2	The GP allocation problem	13
2.a	Definisjon	14
2.b	Løsninger	15
2.c	Leksikografisk optimalitet	16
2.d	Kardinalitet	18
2.e	Nytte (utility)	19
2.f	Prioritetsfunksjon	21
3	Algoritmer	22
3.a	Top Trading Cycles (TTC)	23
3.b	Greedy DFS (heuristikk)	24

Contents (ii)



3.c	Cycle Cancelling	26
3.d	Cycle Cancelling for kardinalitet	27
3.e	Cycle Cancelling for Nytte	28
3.f	Cycle Cancelling for leksikografisk optimalitet	29
3.g	Egenskaper	30
4	Eksperimenter og resultater	31
4.a	Simulering og data	32
4.b	Metrikker	33
4.c	Resultater: Antall hjulpet	34
4.d	Resultater: Ventelistestørrelse	35
4.e	Resultater: Ventetid	36
4.f	Resultater: Kjøretid	37
4.g	Resultater:	38
4.h	Konklusjon	39
4.i	Videre arbeid	40
4.j	Takk for meg!	41

1 Intro

Motivasjon



- Fastlegeordningen (2001)
- Lang kø
 - Bergen 97%
 - 274 dager i 2024
- Ansetter flere fastleger

Motivasjon (ii)

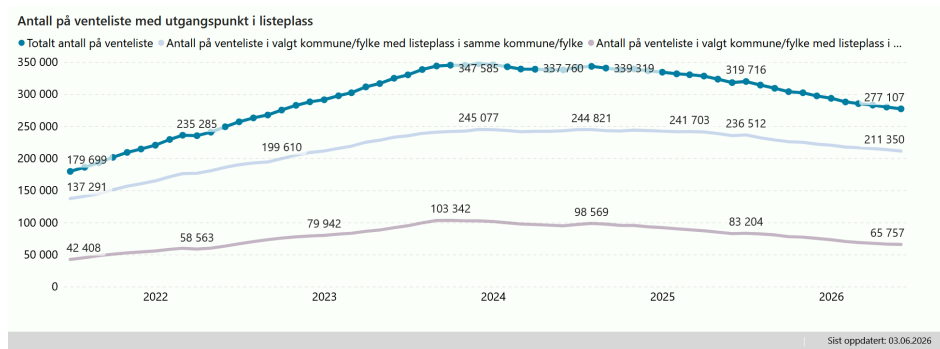


Figure 1: Antall på venteliste i norge, Kilde: Helfo (2024). *Fastlegestatistikk.helfo.no* (lest 21.06.2026).

Løsningen?



nevn dette med hvordan køene fungerer til nå

- Redusere antall folk på venteliste -> Flere folk er fornøyd?

Løsningen? (ii)

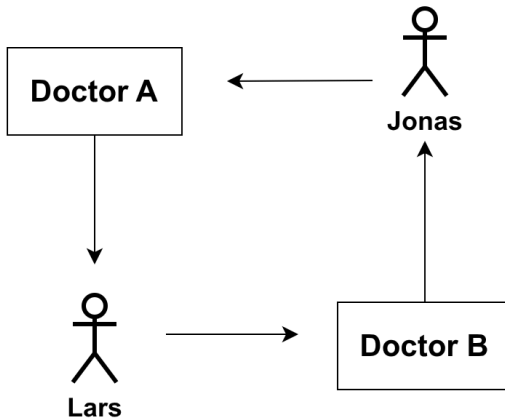


Figure 2: Eksempel med Lars og Jonas på venteliste til hverandres doktor

Bakgrunn



- Agenter og Objekter
- Lignende problemer
 - Housing Market
 - Kidney exchange
 - College Admissions problem
- Top Trading Cycles algoritmen

Top Trading Cycles (TTC)



- $A = [h_B, h_C, h_A]$
- $B = [h_C, h_A, h_B]$
- $C = [h_B, h_C, h_A]$

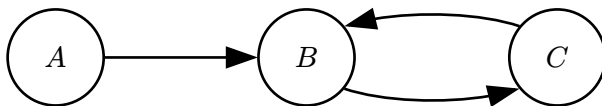


Figure 3: Eksempel ttc graf.

Top Trading Cycles (TTC) (ii)



- $A = [h_B, h_C, h_A]$
- $B = [h_C, h_A, h_B]$
- $C = [h_B, h_C, h_A]$

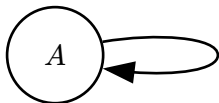


Figure 4: An example Housing Market for TTC, A's Top is itself.

- Strategi-trygt
- Individuell rasjonalitet
- Pareto-effektiv

Huitfeldt et al. TTC



- Pasienter har prioritet
 - Ventetid
- Flere pasienter kan eie samme ønskede fastlege $\rightarrow$ Top(i) ikke entydig
- Først et ventelistesteg (fyller ledige slots), så TTC på resten
- Mister strategitrygghet når den kjøres gjentatte ganger

2 The GP allocation problem

Definisjon



- Input:
 - Pasienter P , fastleger D
 - $D_{\text{cur}[i]}$ – nåværende fastlege, $D_{\text{pref}[i]}$ – ønsket fastlege
 - Prioritetsfunksjon $R : P \rightarrow \mathbb{N}$ (høyere = viktigere)
- Mulig løsning (feasible):
 - Pasientene i S bytter slots samtidig
 - Alle i S ender hos sin ønskede fastlege
 - Antall pasienter per fastlege er uendret
- Kant (a, b) : pasient a kan overta plassen til b
- S er mulig $\Leftrightarrow G_S$ dekkes av vertex-disjunkte sykler

Løsninger



- Når er én løsning bedre enn en annen?
- En ordning $\succ$ over mulige løsninger
- Optimal: ingen mulig løsning er større under $\succ$
- Tre varianter:
 - ▶ Leksikografisk – etter prioritet
 - ▶ Kardinalitet – etter antall
 - ▶ Nytte (utility) – en kombinasjon

Leksikografisk optimalitet



- Pasienter sortert etter prioritet (høyest først), $R(p_i) = i$
 - Karakteristisk vektor $\chi(S) \in \{0, 1\}^n$
 - $S \succ_{\text{lex}} S'$: $\chi(S)$ er leksikografisk større enn $\chi(S')$
 - Som et binærtall – mest signifikante bit = høyest prioritet som hjelpes
 - Hjelp alltid den høyest prioriterte du kan, så nest høyeste, ...
 - Tie-break avgjør ved lik prioritet
-
- Eksempel: $S = \{p_4, p_5\}$ vs. $S' = \{p_1, p_2, p_3\}$
 - $\chi(S) = (1, 1, 0, 0, 0)$, $\chi(S') = (0, 0, 1, 1, 1) \rightarrow S \succ_{\text{lex}} S'$

Leksikografisk optimalitet (ii)

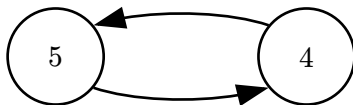
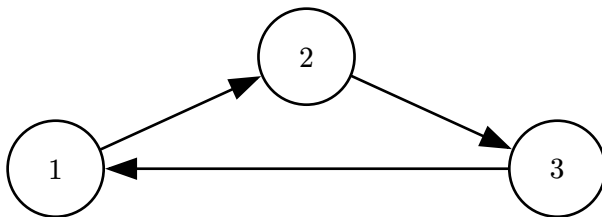


Figure 5: An example graph G.

Kardinalitet



- Flest mulig pasienter byttet
- $S \succ_{\text{size}} S'$ hvis $|S| > |S'|$
- Ingen hensyn til prioritet – alle teller likt

- Eksempel (samme graf):
 - ▶ $|S| = 2, |S'| = 3 \rightarrow S' \succ_{\text{size}} S$
 - ▶ Optimalt: $\{p_1, p_2, p_3, p_4, p_5\}$ (begge sykler)
- Merk: $S \succ_{\text{lex}} S'$ men $S' \succ_{\text{size}} S$ – motsatt rekkefølge!

Nytte (utility)



- Mellomting mellom leksikografisk og kardinalitet
 - Total nytte: $U(S) = \sum_{p \in S} R(p)$
 - $S \succ_{\text{util}} S'$ hvis $U(S) > U(S')$
 - Hver pasient teller, vektet med prioritet
-
- Eksempel: $U(S) = 1 + 3 + 4 = 8$ vs. $U(S') = 2 + 5 = 7 \rightarrow S \succ_{\text{util}} S'$
 - Mange lavprioriterte kan slå få høyprioriterte

Nytte (utility) (ii)

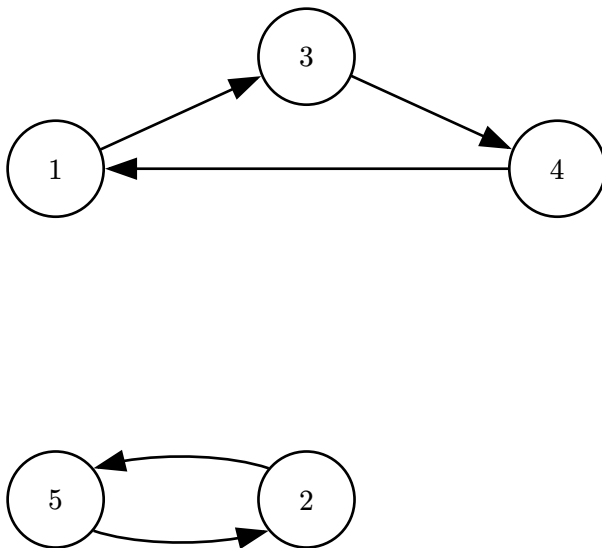


Figure 6: An example graph G.

Prioritetsfunksjon



- Stor effekt på hvilke pasienter som faktisk hjelpes
- $R(a) = k^{\text{dager ventet}}$, med $1 \leq k \leq 2$
- $k = 1$: alle får prioritet 1 $\rightarrow$ nytte = kardinalitet
- $k = 2$: høyprioritert slår alle under seg $\rightarrow$ nytte $\approx$ leksikografisk

3 Algoritmer

Top Trading Cycles (TTC)



- Baseline vi sammenligner mot
- Direkte port fra Python til Rust (lik kjøretidssammenligning)
- Alle fastleger fulle → ventelistesteget hopper over

Greedy DFS (heuristikk)



- Rask og enkel heuristikk som fortsatt respekterer prioritet
- Inspirert av TTC, men lar fastlegen peke til ALLE pasientene sine
- Gå gjennom pasienter i synkende prioritet, kjør DFS for å finne en sykel
- Sykel funnet $\rightarrow$ løs den med en gang og fjern pasientene
- Grådige valg $\rightarrow$ ikke garantert optimal
- $O(|P| (|D| + |P|))$

Greedy DFS (heuristikk) (ii)

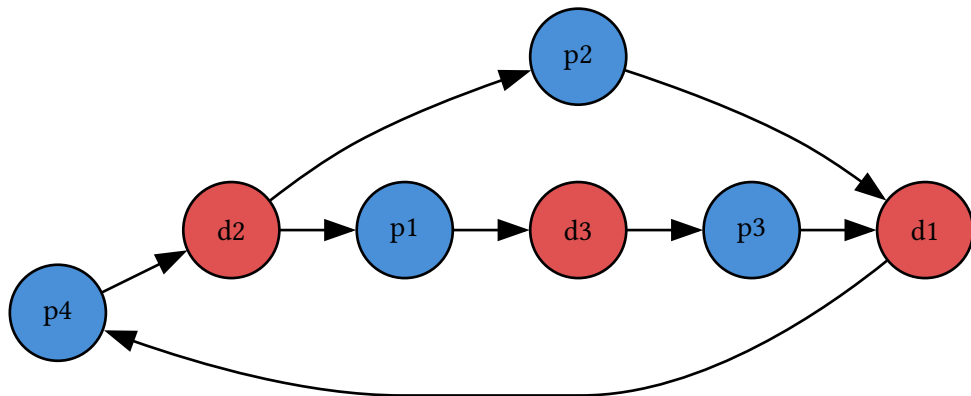


Figure 7: Example graph G where Greedy DFS would choose suboptimal solution. px means patient x and has priority x .

Cycle Cancellation



- Teknikk for minimum-cost circulation
- Residualnettverk – ledig kapasitet + flyt som kan angres
- Negative Cycle Optimality Theorem:
 - Optimal $\Leftrightarrow$ ingen negativ sykel i residualnettverket
- Algoritme: start $f = 0$; så lenge negativ sykel finnes $\rightarrow$ kanseller den
- Bellman-Ford finner syklene, $O(nm^2CU)$ (pseudo-polynomisk)

Cycle Cancelling for kardinalitet

- GP collapsed graph – kun fastleger som noder
- Hver kant: kostnad -1 , kapasitet = antall pasienter som vil bytte
- Min kostnad = mest flyt = flest pasienter
- Polynomisk: $C = 1, U \leq |P| \rightarrow O(nm^2 |P|)$

Cycle Cancellation for Nytte



- GP multigraph – hver pasient er én egen kant
- Kapasitet 1, kostnad $-R(p_i)$
- Min kostnad = optimal under $\succ_{\text{util}}$
- Kjøretid $O(nm^2 \max R(p))$ – polynomisk hvis prioritet er polynomisk begrenset
- Eksponentiell prioritet: 128-bit heltall, buckets på 10 dager

Cycle Cancelling for leksikografisk optimalitet

- GP multigraph, men uten kostnader
- Løp gjennom pasienter i synkende prioritet
- Finnes sti $D_{\text{pref}[i]} \rightarrow D_{\text{cur}[i]}$ i residualgrafene?
 - Ja $\rightarrow$ push én enhet flyt, legg pasienten til i løsningen
- Slett kanten $\rightarrow$ beslutningen er endelig
- Monotonicity of Feasibility – hvorfor det er trygt å slette
- $O((n + m) |P|)$ – raskere enn Bellman-Ford, håndterer tie-breaks eksakt

Egenskaper



- Individuell rasjonalitet: alle algoritmene (ingen flyttes dit de ikke vil)
- Pareto-effektiv: ingen kan forbedres uten å forverre en annen
- Strategitrygghet:
 - ▶ Én kjøring: ja (sannferdig rapportering er dominant)
 - ▶ Gjentatt: usikkert – prioritet avhenger av ventetid

4 Eksperimenter og resultater

Simulering og data



- Syntetiske data (personvern + tid), proporsjoner matchet til Norge
- 100 000 pasienter, 102 fastleger (980 per lege)
- 5% startventeliste, 6 distrikter (17 leger hver)
- 11% forespørsler på tvers av distrikt, 18 nye forespørsler/dag
- Dag for dag:
 - Prioritet og ventetid +1
 - Kjør algoritmen → flytt løste pasienter
 - Legg til nye forespørsler
- 100 år (36 500 dager) + 2 år for de eksponentielle variantene

Metrikker



- Antall hjulpet
- Størrelse på ventelisten
- Vente tid
 - Maks
 - Gjennomsnitt
 - 99-persentil
- Andel på tvers av distrikt
- Kjøretid

Resultater: Antall hjulpet



- Alle algoritmene løser nesten like mange (1% forskjell)
 - 622 002 (strict priority) – 629 497 (cardinality)
- Spørsmålet er ikke **hvor mange**, men **hvilke**

Resultater: Ventelistestørrelse



- Cardinality lavest, strict priority højest
- Overraskelse: Huitfeldt TTC nesten på linje med cardinality
- Listene klatrer hele tiden – lukket modell (se forbehold)

Resultater: Ventetid



- Cardinality & Huitfeldt TTC: kortest snitt, men noen venter nesten “evig”
- Strict priority: lengst snitt, men best verste-tilfelle
- Snitt: Huitfeldt 352.9 d, cardinality 940.7 d, resten > 1300 d
- Maks: cardinality & TTC når 36 500 d (aldri løst); resten er begrenset
- 99-persentil: cardinality verst (14 219 d), strict priority best (9 335 d)

Resultater: Kjøretid



- Cardinality klart raskest: 9.2 ms/dag
- Resten: 128–379 ms/dag (én grafsøk per pasient → tregest)
- Grunn: collapsed graph skalerer med antall fastleger, ikke pasienter

Resultater:



- Samme antall hjulpet – men ulike pasienter
- Gjennomstrømning (kort snittventetid) vs. rettferdighet (begrenset maks)

Konklusjon



- Formaliserte GP allocation-problemet som sykler i en rettet graf
- Tre optimalitetskriterier – nytte har kardinalitet og leksikografisk som endepunkter
- Eksakte cycle cancelling-algoritmer + rask Greedy DFS-heuristikk
- Hovedfunn: valg av mål endrer nesten ikke **hvor mange**, men **hvilke** – og dermed ventetiden
- Avveining: gjennomstrømning vs. verste-tilfelle-rettferdighet
- Uventet: TTC-baselen holder følge med cardinality på ventelistestørrelse

Videre arbeid



- Ekte data fra Helfo
- Åpen modell med ledig kapasitet + et ventelistesteg før byttene
- Gjentatte kjøringer med konfidensintervall
- Strategisk oppførsel i gjentatt setting
- Sterkt polynomisk metode (Goldberg–Tarjan) / begrenset sykellengde / topartspreferanser

Takk for meg!

- Spørsmål?

